Procesos de Desarrollo de Software

CLASE 10 – PATRONES DE DISEÑO

Agenda de la clase

- Composite
- Decorator

Composite 🌳

Es un patrón **Estructural.**

¿Qué hace?

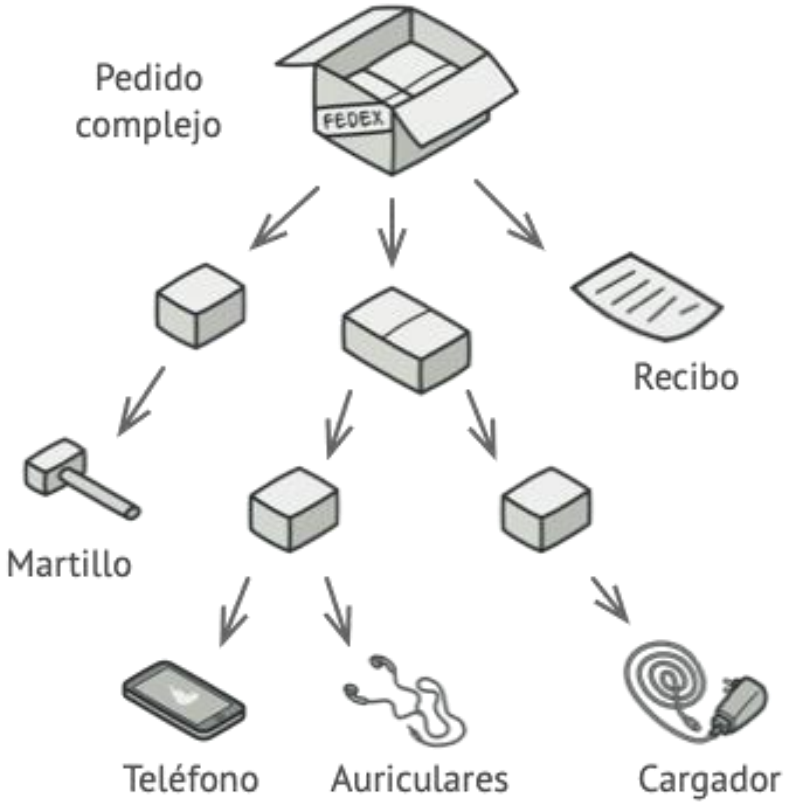
- Compone objetos en estructura de árbol.
- Permite que un cliente trate de forma polimórfica a un objeto particular y a un conjunto combinado de ellos.

Se **sugiere** su utilización **cuando**:

- Se requiere que un tercero utilice de forma indistinta (polimórfica) a un objeto particular o a un conjunto compuesto de ellos.
- Se debe permitir que un objeto esté compuesto por varios objetos simples o por otros objetos compuestos.

Ejemplo

- Dentro del pedido hay:
  - productos individuales
  - otros grupos de productos



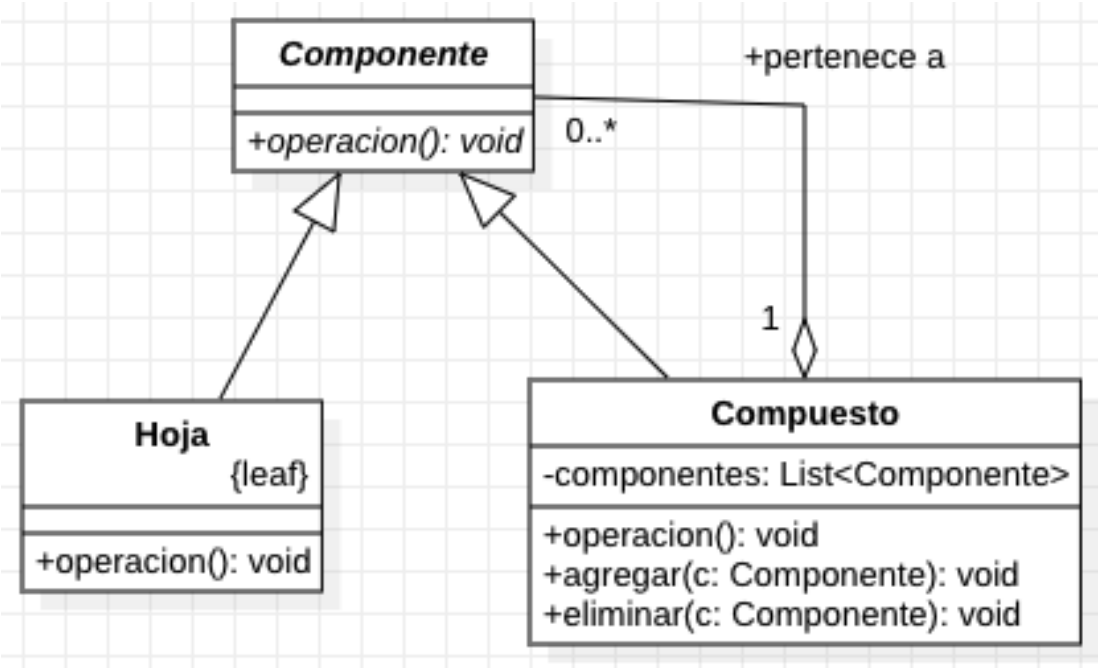
Componentes

Componentes:

- **Interface/Clase abstracta Componente**: Define las firmas de los métodos que son comunes entre los elementos simples y los elementos complejos/compuestos. Su propósito es que un tercero conozca su interfaz para que utilice polimórficamente los elementos simples y los compuestos.
- **Hoja**: Objetos simples que cumplen con la interfaz del Componente. Contienen la implementación más "gruesa" ya que, en general, no delegan el trabajo en nadie más.
- **Compuesto**: Clase que, también, cumple con la interfaz del Componente. Contiene una colección de Componentes, motivo por el cual puede contener tanto objetos simples (hojas) como otros objetos compuestos. Al recibir un mensaje, delega el trabajo en los componentes que tiene; luego junta las respuestas, las procesa y, por último, otorga una respuesta final simplificada.

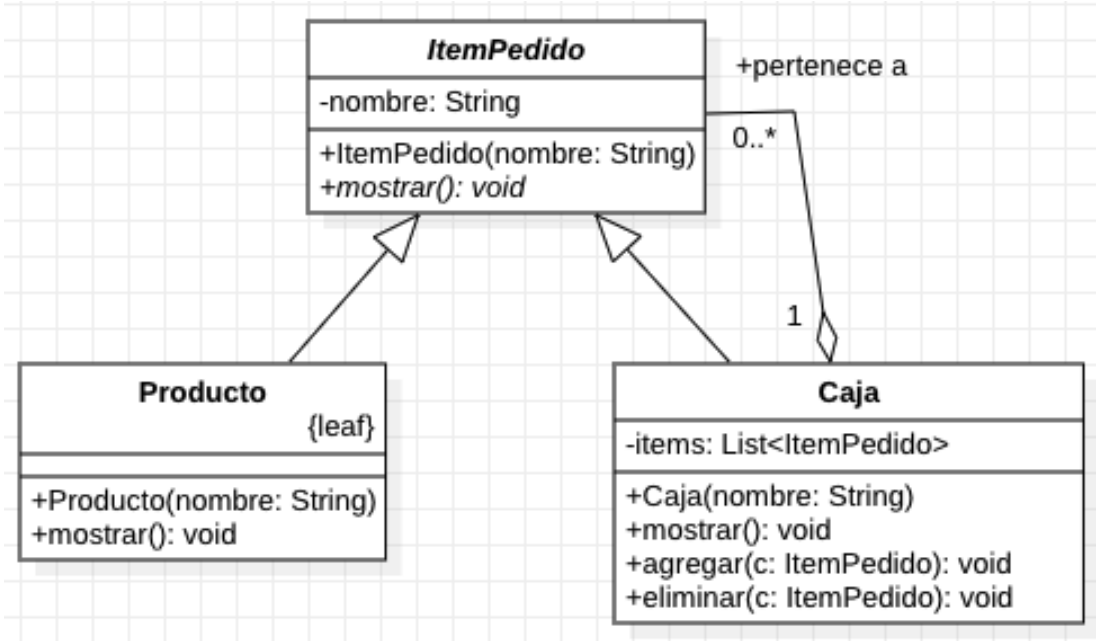
Diagrama UML

- "Un objeto contiene otros objetos del mismo tipo lógico"



Ejemplo

- Un pedido puede tener dentro varias ‘cajas’ con distintos ítems.



Ejemplo

```java
public abstract class ItemPedido {
    protected String nombre;

    public ItemPedido(String nombre) {
        this.nombre = nombre;
    }

    public abstract void mostrar();
}
```

```java
public class Producto extends ItemPedido {

    public Producto(String nombre) {
        super(nombre);
    }

    @Override
    public void mostrar() {
        System.out.println(nombre);
    }
}
```

```java
public class Caja extends ItemPedido {

    private List<ItemPedido> items =
            new ArrayList<>();

    public Caja(String nombre) {
        super(nombre);
    }

    public void agregar(ItemPedido item) {
        items.add(item);
    }

    @Override
    public void mostrar() {
        System.out.println("Caja: " + nombre);

        for(ItemPedido item : items) {
            item.mostrar();
        }
    }
}
```

Práctica

- Ejercicio de aplicación de patrones "**Módulo de Stock y Precios**"
- Ejercicio de aplicación de patrones "**Módulo de Restaurante**"

Magic Café ☕ ✨

- Antes el pedido solamente podía vender sólo Producto
- **Problema**: No podíamos vender combos reutilizables.

- Queremos permitir:
  - Productos simples
  - Combos
  - Combos dentro de combos

Magic Café ☕ ✨

- De vender solo productos simples a vender productos, combos y estructuras reutilizables.
- La UI Swing y Clean Architecture se mantienen

Evolución de la clase

JPA/Hibernate
↓
JpaRepository
↓
Strategy persistida
↓
Composite para combos

Objetivo

- Entender qué problema resuelve Composite.
- Identificar Component, Leaf y Composite en MagicCafe.
- Modificar el dominio para trabajar contra una abstracción común.
- Persistir Producto y Combo con Hibernate usando herencia JPA.
- Ver cómo Pedido puede vender cualquier Vendible sin if/else.

Punto de partida

- La aplicación ya tiene UI, Controller, Service, Repository y Hibernate.



Limitación

El modelo solo pensaba en Producto. Si queríamos vender un combo, debíamos inventar lógica especial o crear una clase separada difícil de integrar.

Problema de diseño

Antes

```
public class Pedido {

    public void agregarProducto(
            Producto producto,
            int cantidad) {
        ...
    }
}
```

Pedido queda atado a Producto.

Nueva necesidad

Vender café individual.

Vender combo desayuno.

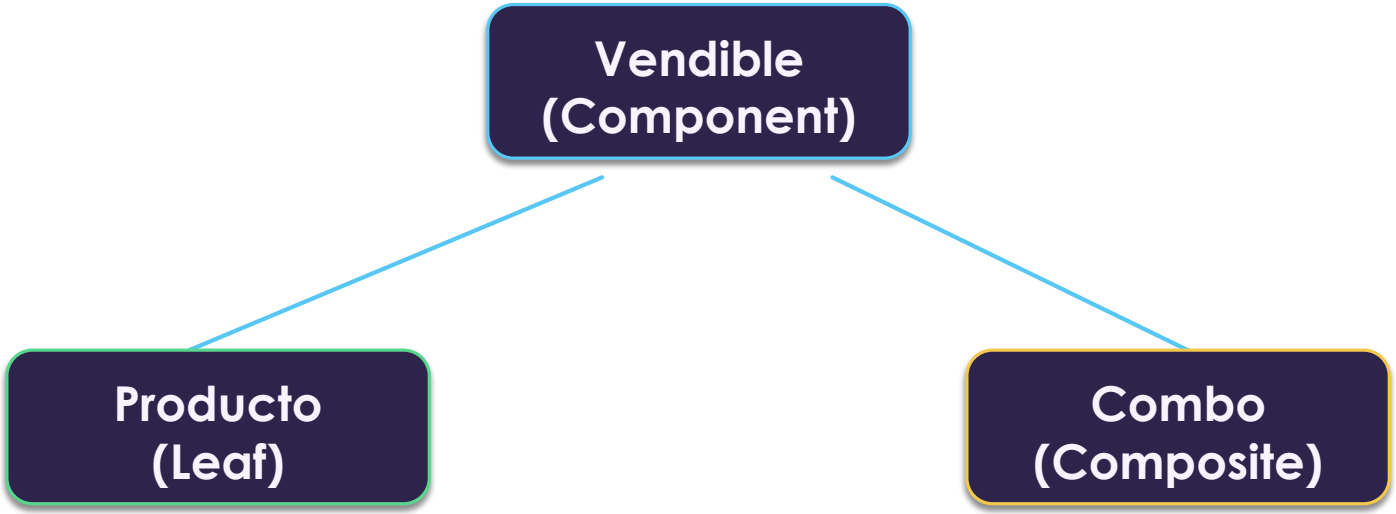
Vender combo merienda.

Permitir combos formados por otros vendibles.

No queremos llenar Pedido de if/else.

Solución

- La pregunta clave deja de ser “¿es un producto?” y pasa a ser “¿se puede vender?”
- Producto y Combo comparten el mismo contrato: calcularPrecio()



Paso1: crear el componente - Vendible

- Vendible representa cualquier cosa que MagicCafe puede vender.

```
@Entity
@Table(name = "vendibles")
@Inheritance(strategy = InheritanceType.SINGLE_TABLE)
@DiscriminatorColumn(name = "tipo_vendible")
public abstract class Vendible {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    protected String nombre;

    public abstract double calcularPrecio();
}
```

Qué aporta

Contrato común.

Permite polimorfismo.

Evita depender de Producto.

Es persistible con Hibernate.

Paso 2: Producto como Leaf

- Producto es una hoja: no contiene otros elementos.

```
@Entity
@DiscriminatorValue("PRODUCTO")
public class Producto extends Vendible {

    private double precio;

    public Producto(String nombre, double precio) {
        super(nombre);
        this.precio = precio;
    }

    @Override
    public double calcularPrecio() {
        return precio;
    }
}
```

Rol en Composite

Producto = Leaf

Es el caso más simple: sabe calcular su propio precio y no tiene hijos.

Paso 3: Combo como Composite

- Combo también es Vendible, pero está compuesto por otros Vendible.

```
@Entity
@DiscriminatorValue("COMBO")
public class Combo extends Vendible {

    @OneToMany(mappedBy = "combo",
               cascade = CascadeType.ALL,
               orphanRemoval = true,
               fetch = FetchType.EAGER)
    private List<ComboItem> items = new ArrayList<>();

    @Override
    public double calcularPrecio() {
        return items.stream()
            .mapToDouble(ComboItem::calcularSubtotal)
            .sum();
    }
}
```

Rol en Composite

Contiene hijos.

Reutiliza Vendible.

Calcula precio recorriendo items.

Puede contener productos o combos.

Paso 4: ComboItem une Combo con Vendible

- ComboItem representa una línea interna dentro de un combo.

```
@Entity
@Table(name = "combo_items")
public class ComboItem {

    @ManyToOne(fetch = FetchType.LAZY)
    @JoinColumn(name = "combo_id", nullable = false)
    private Combo combo;

    @ManyToOne(fetch = FetchType.EAGER)
    @JoinColumn(name = "vendible_id", nullable = false)
    private Vendible vendible;

    private int cantidad;

    public double calcularSubtotal() {
        return vendible.calcularPrecio() * cantidad;
    }
}
```

Ejemplo

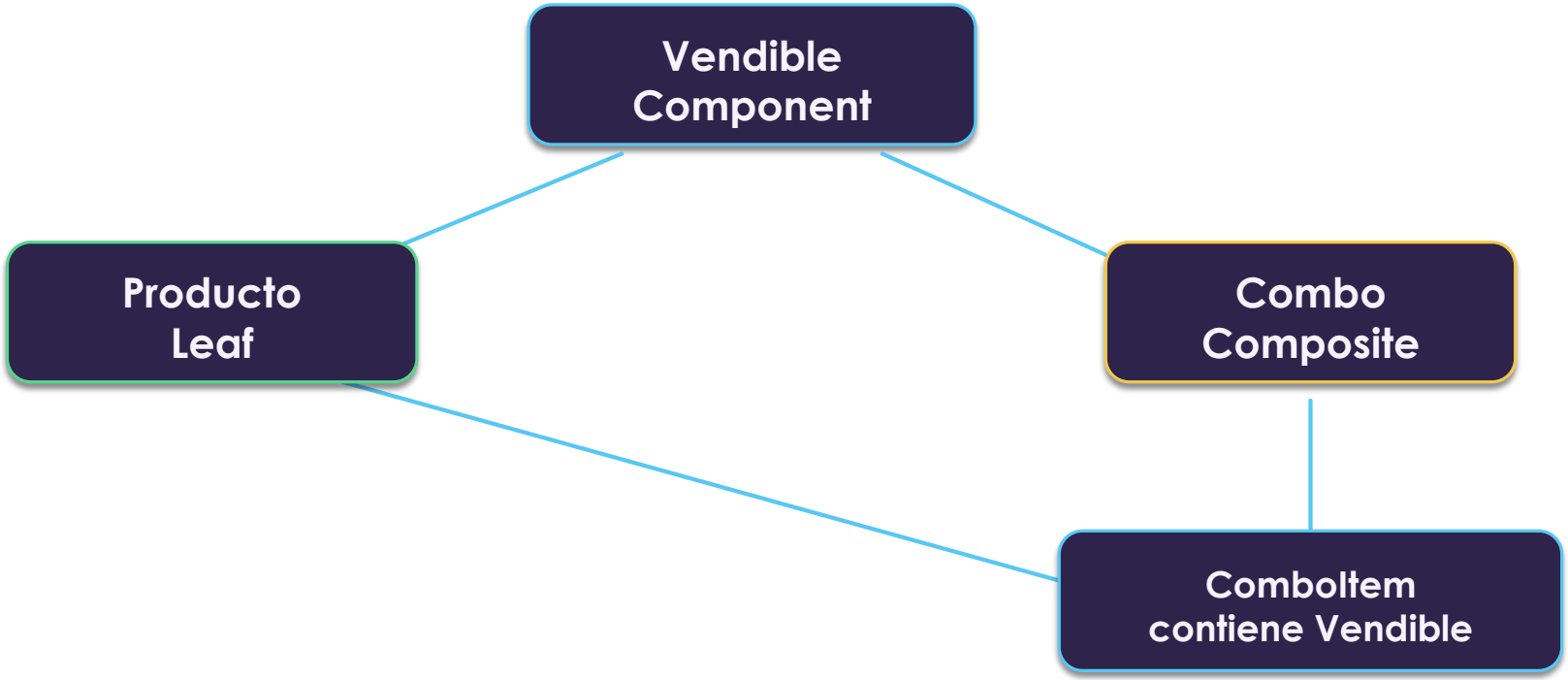
Combo Desayuno

1 × Café Latte
2 × Croissant

Cada línea es un ComboItem.

Estructura completa del Composite

- El combo puede apuntar a productos o a otros combos porque ambos son Vendible.



Pedido ahora debe usar Vendible

Antes

```
public class Pedido {

    public void agregarProducto(
            Producto producto,
            int cantidad) {
        ItemPedido item =
            new ItemPedido(this, producto,
cantidad);
    }
}
```

Pedido depende de Producto.

Ahora

```
public class Pedido {

    public void agregarVendible(
            Vendible vendible,
            int cantidad) {
        ItemPedido item =
            new ItemPedido(this, vendible,
cantidad);
    }
}
```

Pedido depende de una abstracción.

ItemPedido también usa Vendible

- La línea del pedido puede ser un producto simple o un combo.

```
@Entity
@Table(name = "pedido_items")
public class ItemPedido {

    @ManyToOne(fetch = FetchType.EAGER)
    @JoinColumn(name = "vendible_id", nullable = false)
    private Vendible vendible;

    private int cantidad;
    private double precioUnitario;
    private double subtotal;

    public ItemPedido(Pedido pedido,
                      Vendible vendible,
                      int cantidad) {
        this.precioUnitario = vendible.calcularPrecio();
        this.subtotal = precioUnitario * cantidad;
    }
}
```

Beneficio

ItemPedido ya no pregunta:

"¿sos Producto o Combo?"

Solo llama:

vendible.calcularPrecio()

La UI no necesita saber demasiado

- El combo de la UI ahora lista VendibleDTO: productos y combos mezclados.

```
private final JComboBox<VendibleDTO>
vendiblesCombo
        = new JComboBox<>();

for (VendibleDTO vendible :
controller.listarVendibles()) {
    vendiblesCombo.addItem(vendible);
}

controller.agregarItem(
        vendible.getId(),
        cantidad
);
```

Resultado en pantalla

[Producto] ☕ Café Latte - $4500

[Combo] ✨ Combo Desayuno - $10900

[Combo] 🍰 Combo Merienda - $10300

Persistencia con Hibernate

- Vendible usa herencia JPA para guardar Producto y Combo en una estructura común.

```
@Entity
@Table(name = "vendibles")
@Inheritance(strategy = InheritanceType.SINGLE_TABLE)
@DiscriminatorColumn(name = "tipo_vendible")
public abstract class Vendible { ... }

@Entity
@DiscriminatorValue("PRODUCTO")
public class Producto extends Vendible { ... }

@Entity
@DiscriminatorValue("COMBO")
public class Combo extends Vendible { ... }
```

Lectura simple

tipo_vendible le dice a Hibernate qué clase reconstruir:

PRODUCTO → Producto
COMBO → Combo

Tablas generadas

- combo_id apunta al Combo. vendible_id apunta al producto o combo contenido.

vendibles

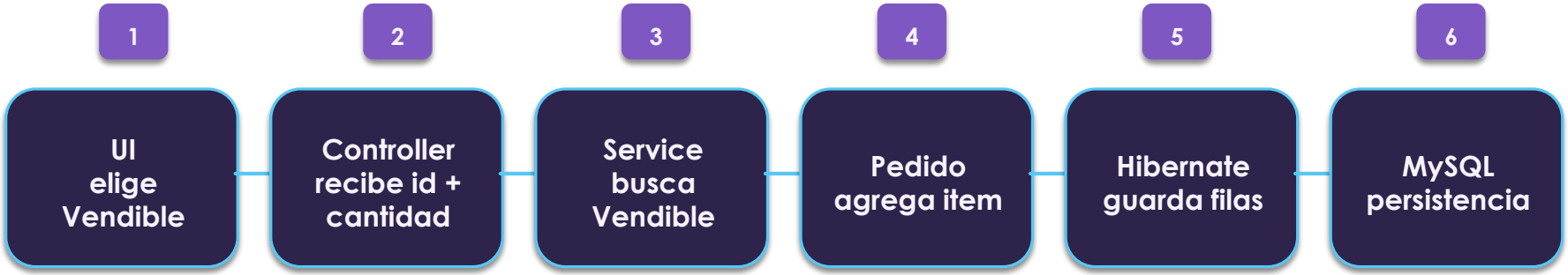
id
tipo_vendible
nombre
precio_producto

combo_items

id
combo_id
vendible_id
cantidad

Flujo al confirmar un pedido con Composite

- En ningún paso el pedido necesita saber si el ítem era Producto o Combo. Solo trabaja con Vendible.



Error frecuente: LazyInitializationException

- Si Combo.items está LAZY y el EntityManager ya cerró, calcularPrecio() puede fallar.

Síntoma

```
failed to lazily initialize a
collection
of role: Combo.items
could not initialize proxy - no
Session
```

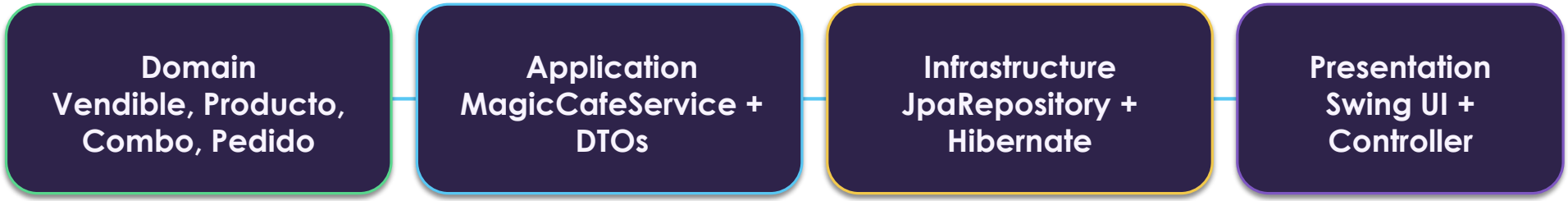
Ocurre al acceder a una relación LAZY fuera del EntityManager.

Solución

```
@OneToMany(
    mappedBy = "combo",
    cascade = CascadeType.ALL,
    orphanRemoval = true,
    fetch = FetchType.EAGER)
private List<ComboItem> items;
```

Clean Architecture

- Composite es una decisión de dominio. No nace por Hibernate ni por Swing. Hibernate solo lo persiste.



Cómo mejora el diseño?

- Pedido depende de Vendible, no de Producto concreto
- Se pueden agregar nuevos tipos vendibles sin tocar Pedido
- Combo reutiliza la misma operación calcularPrecio()
- La UI puede listar productos y combos en el mismo control
- Hibernate reconstruye el subtipo real con tipo_vendible

SOLID

Open/Closed Principle

Abierto para extender:
NuevoVendible

Cerrado para modificar:
Pedido

Level Up!

- Crear un **Combo Premium** que contenga otro **Combo**.
- Agregar un nuevo tipo Vendible: **ProductoEdicionLimitada**.
- Mostrar en la UI si el ítem es Producto o Combo.
- Persistir y verificar las tablas vendibles y combo_items.
- Explicar qué parte del código cambió y cuál no.
- ¿Qué clases **NO** deberían modificarse al agregar un nuevo tipo de Vendible?

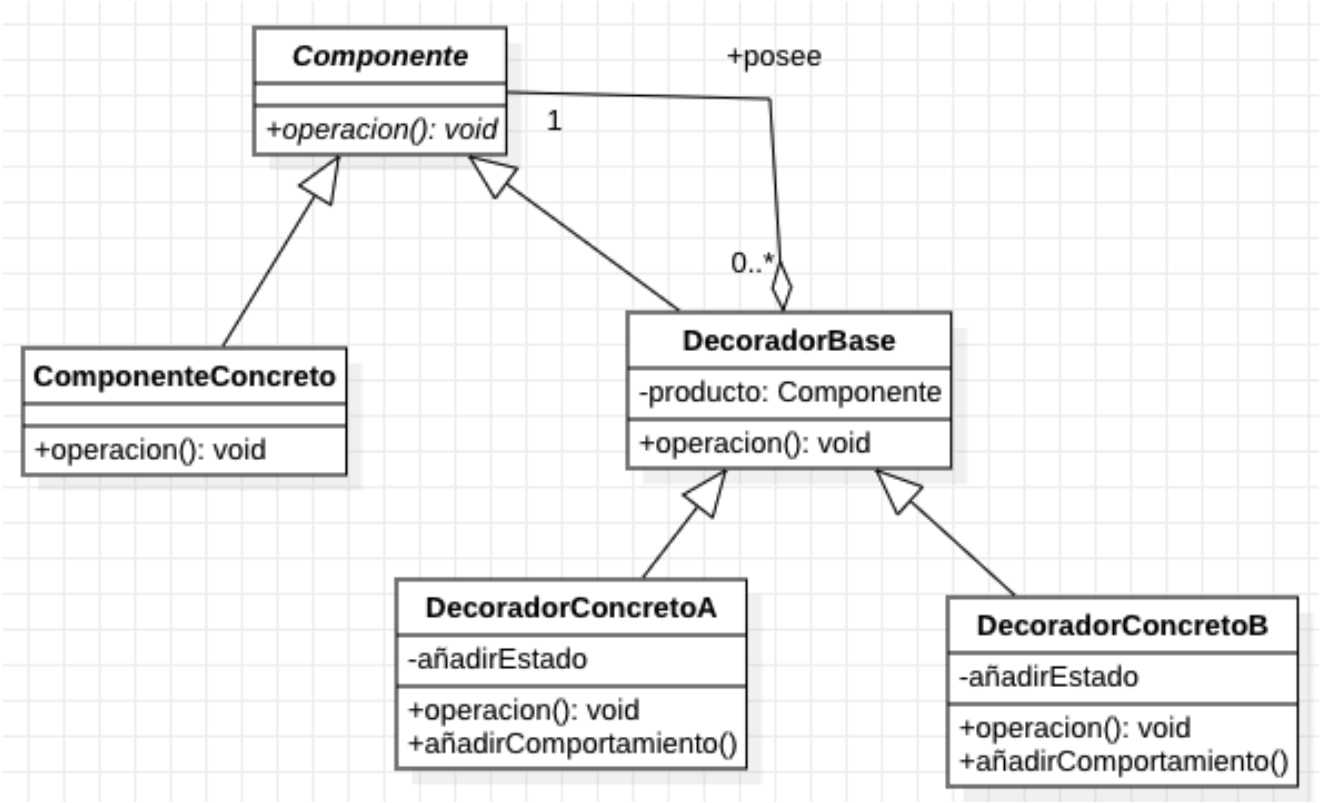
Decorator

- Es un patrón **Estructural.**
- Agrega responsabilidades a un objeto dinámicamente. Es una alternativa flexible a la herencia para añadir funcionalidades.
- **Motivación**
  - Se desea añadir funcionalidad a un objeto concreto, no a una clase entera.
- **Aplicabilidad**
  - Añadir responsabilidades a objetos concretos de manera dinámica y transparente, sin afectar a otros objetos.
  - Cuando las responsabilidades se pueden otorgar o derogar.
  - Cuando la herencia sea muy compleja porque implique la creación de múltiples subclases para dar soporte a todas las combinaciones posibles.

Consecuencias

- Es la más flexible que la herencia estática. Las responsabilidades se añade y eliminan dinámicamente.
  - Facilita definir una propiedad varias veces.
- Evita una complejidad en las clases más altas en las jerarquías.
  - Se facilita la definición de nuevos decoradores
- Un decorador y el componente al que se refieren no son idénticos (tienen identificador distinto)
- Provoca la creación de muchos objetos pequeños parecidos y encadenados, dificultando la depuración

Diagrama UML



Magic Café ☕ ✨

- Aplicando Decorator a productos y combos
- De productos fijos → a productos personalizables dinámicamente



Decorator

- Agregar extras dinámicamente
- Evitar explosión de subclases
- Mantener Composite funcionando
- Aplicar Open/Closed Principle
- Decorator agrega funcionalidades sin modificar las clases originales.

Punto de partida

- Ya tenemos Composite funcionando (Composite, Vendible, Producto, Combo)
- Ahora queremos agregar extras dinámicos.
- Queremos soportar:
  -  Café + leche
  -  Café + crema
  -  Café + canela
- **SIN** crear:
  - CafeConLeche
  - CafeConCrema
  - CafeConLecheYCrema

Solución

ItemVendible
↑
VendibleDecorator
↑
ConLeche / ConCrema / ConCanela

Cada decorator envuelve otro ItemVendible.

```
public interface ItemVendible {

    String getNombre();
    double calcularPrecio();
}
```

- Todos los elementos compartirán este contrato

Vendible implementa ItemVendible

- Producto y Combo ahora son ItemVendible automáticamente.

```
public abstract class Vendible
        implements ItemVendible {

    protected String nombre;
}
```

Decorador Base

- El decorator contiene otro ItemVendible.
- **NO** agrega funcionalidad concreta.
  - **Define la estructura común de todos los decorators.**
  - **Guarda el objeto envuelto.**

```
public abstract class VendibleDecorator
        implements ItemVendible {

    protected ItemVendible vendible;

    public VendibleDecorator(
            ItemVendible vendible) {

        this.vendible = vendible;
    }
}
```

Decoradores concretos

- Agregan comportamiento REAL.

```
public class ConLeche
        extends VendibleDecorator {

    @Override
    public double calcularPrecio() {

        return vendible.calcularPrecio()
                + 300;
    }
}
```

ConCrema y ConCanela

ConCrema
→ agrega 500

ConCanela
→ agrega 150

Todos reutilizan:
vendible.calcularPrecio()

Cómo funcionan?

Cómo se encadenan

```
ItemVendible cafeDecorado =

new ConCanela(
    new ConCrema(
        new ConLeche(cafe)
    )
);
```

Cada decorator envuelve al anterior.

- ☕ Café Latte
  + leche
  + crema
  + canela

  Precio:
  4500 + 300 + 500 + 150

Cómo se crea el Pedido?

- Pedido ya no depende de Producto concreto.

ANTES

```
pedido.agregarProducto(...)
```

DESPUÉS

```
pedido.agregarItem(
        ItemVendible item,
        int cantidad)
```

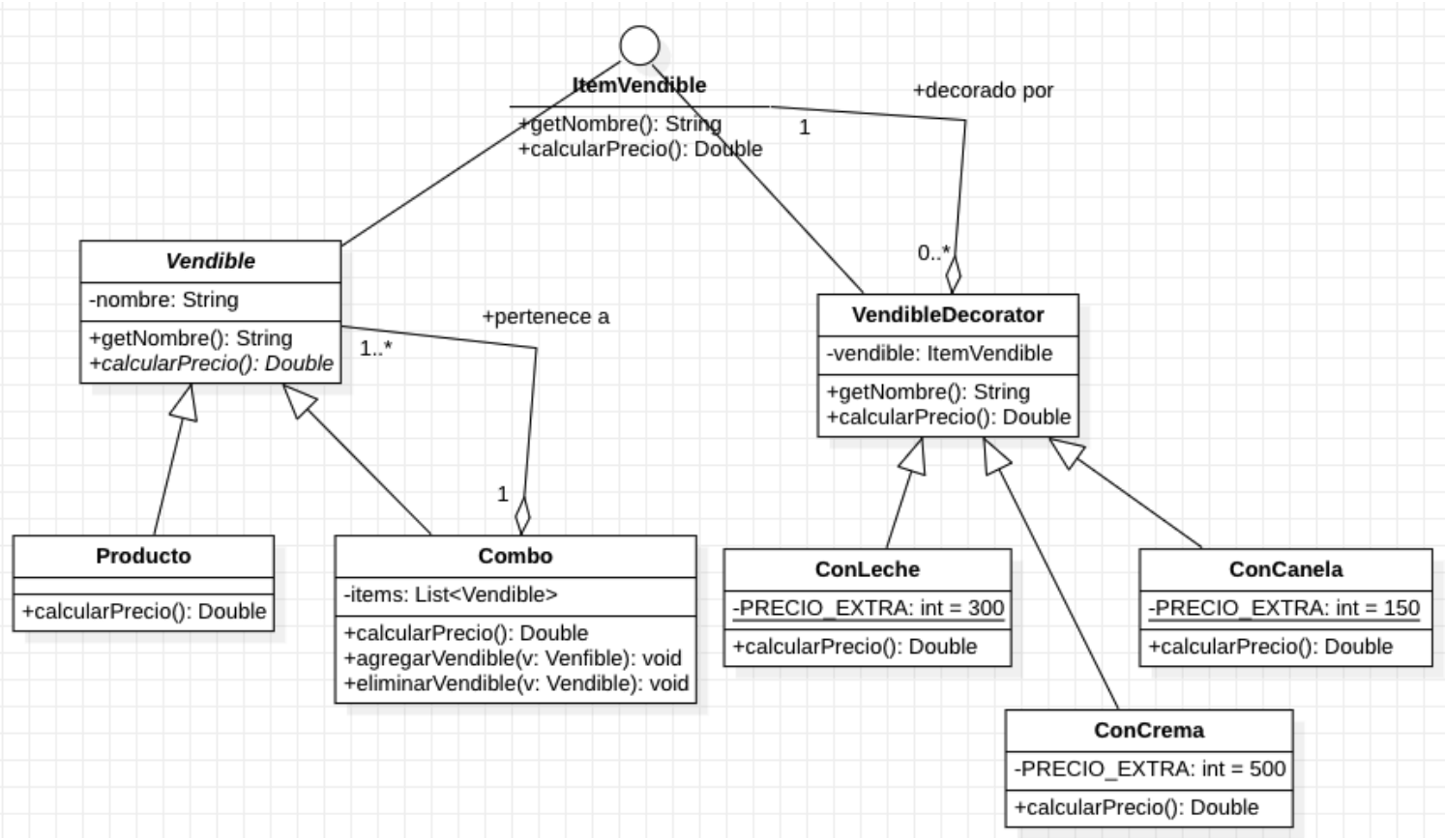
Composite + Decorator 🧩

- Composite
  → Producto y Combo
- Decorator
  → extras dinámicos

Ejemplo:

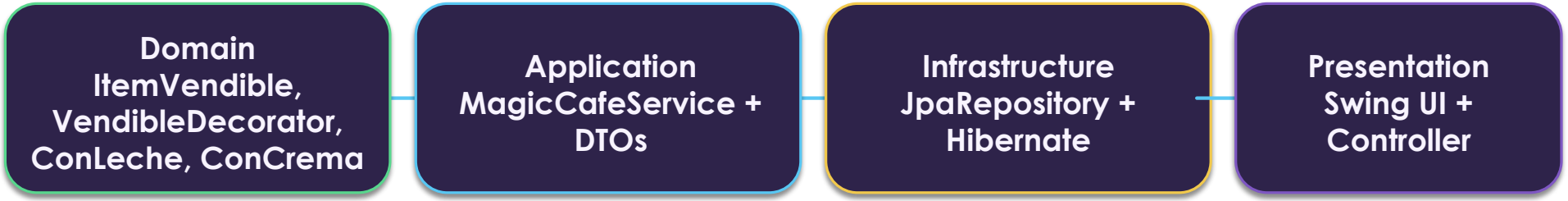
Combo desayuno + crema + canela

Solución UML



Clean architecture

- Decorator vive en el Domain, representa lógica de negocio.



Cómo mejora el diseño?

- **ANTES**
  - Productos fijos
  - Sin extras
  - Muchas subclases
- **DESPUÉS**
  - Decorators encadenables
  - Extras ilimitados
  - Más extensible

Gracias!

- Preguntas?
- mailto: mstricagnoli@uade.edu.ar

